

SOLIDProof

Bring trust into your projects

Blockchain Security | Smart Contract Audits | KYC Development | Marketing

MADE IN GERMANY



Table of Contents

vAPI Staking - Smart Contract Security Audit Report....4
Document Control....4
Revision History....4
Disclaimer....5
Executive Summary....6
Engagement Overview....6
Overall Security Rating....6
Findings Summary....6
Key Observations....7
Scope of Work....8
Target Description....8
Repository & Versioning....8
Reproducibility Metadata....8
Contracts in Scope....9
In Scope....9
Out of Scope....9
Methodology & Tools....11
Approach Overview....11
Techniques Applied....11
Tools Used....11
Review Process....12
Severity & Status Definitions....13
House Standard Risk Model....13
Status Lifecycle....13
Logical Understanding / Contract Architecture....14
System Overview....14
Contract Interactions & Data Flow....14
External Dependencies & Integrations....16
Code Overview....16
Findings Overview....17
Summary Table....17
Detailed Findings....18
STK-05: Deployment inputs are not validated....18
STK-01: Threshold proposals never expire and cannot be withdrawn....20
STK-02: No minimum position size....22
STK-03: Historical lookups accept the current timestamp....24
STK-04: Single-step ownership with an available renounce path....25
STK-06: Interfaces carry no NatSpec....26
STK-07: A custody shortfall is reported as zero....27
STK-08: Unstake requests cannot be cancelled....28
STK-09: Compiler version with known issues....29
Centralisation, Access Control & Privilege Analysis....30
Overview....30
On-Chain Roles & Permissions....30
Upgradeability Analysis....30
Centralisation Risk Summary....30
Conclusion....31
Summary....31
Positive Observations....31
Appendix A - Gas Optimizations and Code Quality Notes....32
Gas Optimizations....32
Code Quality Notes....32

Appendix B - Claims Check.....33
Appendix C - Analysis Coverage by Category.....34

vAPI Staking - Smart Contract Security Audit Report

Field	Value
Report Title	Smart Contract Security Audit - vAPI Staking
Audit Type	Smart contract audit and remediation verification
Version	v1.2
Date	2026-10-19
Classification	Public
Prepared by	SolidProof
Prepared for	vAPI

Document Control

Revision History

Version	Date	Changes
v1.0	2026-10-13	Initial release
v1.1	2026-10-14	Peer review integrated; STK-01, STK-02 and STK-05 reclassified
v1.2	2026-10-19	Remediation verified; all nine findings Verified

Disclaimer

This report is provided on an "as-is" basis and reflects the state of the smart contracts at the time of the audit. It does not constitute legal, financial, or investment advice.

Limitations:

- Security audits are time-boxed engagements and cannot guarantee the absence of all vulnerabilities.
- This audit does not replace other security measures such as bug bounty programs, on-chain monitoring, incident response planning, or ongoing security reviews.
- Findings are based on the code, configurations, and documentation provided by the client at the time of engagement. Changes made after the audit may introduce new risks.
- The assessment does not cover economic, game-theoretic, or market manipulation attack vectors unless explicitly stated in the scope.
- Third-party dependencies and integrations are assumed to function as documented unless explicitly included in the audit scope.
- This report is published with the consent of vAPI.

Liability:

SolidProof assumes no liability for any losses, damages, or exploits arising from the use of the audited smart contracts, whether or not the vulnerabilities were identified in this report.

Executive Summary

Engagement Overview

SolidProof was engaged by vAPI to audit the staking contracts `VapiStaking` and `ThresholdReviewersGate`, their interfaces and the deployment script `DeployVapiStaking.s.sol`.

- **Initial audit:** 2026-10-13 to 2026-10-14 on commit `01771b34ac625baa6ba5a75273cceef3e1dcc044`.
- **Remediation verification:** the client delivered fixes for all nine findings on 2026-10-18, and SolidProof verified them on 2026-10-19 on commit `716796dcf`.
- **Escrow stack:** the Tasks escrow stack, which uses the gate for reviewer eligibility, is covered by a separate report.

Overall Security Rating

Rating	Description
Critical	Contracts are at risk of exploitation
Needs Improvement	Significant issues, fix before deployment
Moderate	Some issues, manageable with timely fixes
Secure	No critical or high issues
Excellent	Robust security posture

This engagement's rating: Secure

No critical, high or medium severity issue was found. The low severity finding and all eight informational findings have been fixed and independently verified. The remediation introduced no new vulnerability.

Findings Summary

Severity	Count	Open	In Progress	Verified	Accepted Risk
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	1	0	0	1	0
Informational	8	0	0	8	0
Total	**9**	**0**	**0**	**9**	**0**

Four optional gas optimizations (STK-G1 to STK-G4) are listed in Appendix A. They are not counted above and were not addressed.

Key Observations

- **All nine findings are fixed and verified.** The deployment script validates the token, its decimals and the owner Safe before anything is broadcast, and the record is written from confirmed receipts (STK-05). The gate has expiring and cancellable threshold proposals (STK-01) and two-step ownership with renouncing disabled (STK-04). Staking enforces a minimum position of one vAPI (STK-02), rejects lookups of the current second (STK-03), reports a custody deficit (STK-07) and supports cancelling an unstake request (STK-08). The interfaces are fully documented (STK-06), and the staking scope uses Solidity 0.8.37 (STK-09).
- **Custody guarantees still hold.** All 18 symbolic specifications, updated for the new rules, passed. Twelve stateful properties held across 300,286 Echidna calls and 9,034,531 Medusa calls.
- **Real-token tests executed.** We ran all seven fork tests against public Base RPC endpoints: four on Base Sepolia and three against the real vAPI token on a Base mainnet fork. All passed.
- **Open before release.** The announced deployment commit `3ed73a7d3`, which changes the initial threshold to 10,000 vAPI, and the deployed bytecode have not been verified yet (Appendix I).

Scope of Work

Target Description

`VapiStaking` is a custody contract for the existing vAPI ERC-20 token:

- Holders stake at least one vAPI.
- An unstake request stops counting immediately and can be cancelled at any time before withdrawal.
- The same principal is withdrawn to the same wallet after a fixed seven-day cooldown.
- Every balance change writes OpenZeppelin `Trace208` checkpoints keyed by timestamp. The off-chain Compute control plane reads them at 00:00 UTC every day to assign Router usage limits.

`ThresholdReviewersGate` answers `stakedBalance(account) >= threshold` for the Tasks reviewer registry:

- In the verified commit the threshold starts at 100,000 vAPI, bounded between 10,000 and 1,000,000 vAPI.
- It is owned by the Foundation multisig through two-step ownership that cannot be renounced.
- A threshold change needs a seven-day proposal followed by a 14-day execution window, and the owner can cancel a pending proposal.

Repository & Versioning

Field	Value
Repository	vAPI monorepo (private), `contracts/` Foundry project
Audited commit	`01771b34ac625baa6ba5a75273cceef3e1dcc044` (v1.0, v1.1)
Remediated commit	`716796dcf`, branch `router-staking-v1` (v1.2)
Compiler	Solidity 0.8.37, via-IR, optimizer 200 runs, EVM cancun
Framework	Foundry (forge 1.7.1)
Chain/Network	Base mainnet (8453), not yet deployed

The remediation package matches the audited code exactly before the change and the delivered sources after it.

Reproducibility Metadata

Field	Value
Environment	macOS 15.8.1, arm64
Toolchain	forge 1.7.1, solc 0.8.37 (`f401782d`), Slither 0.11.5, Aderyn 0.6.8, Halmos 0.3.3, Echidna 2.3.3, Medusa 1.5.1
Dependencies	OpenZeppelin Contracts 5.6.1, forge-std v1.16.1 (vendored)

Field	Value
Analysis window	2026-10-13 to 2026-10-14 (audit), 2026-10-19 (verification)
Assumptions	vAPI is a plain ERC-20 with 18 decimals, no fees or hooks; Compute not reviewed

Contracts in Scope

#	File	LoC	Description
1	`src/VapiStaking.sol`	250	Custody, minimum stake, cooldown, cancellation, checkpoints
2	`src/ThresholdReviewersGate.sol`	172	Reviewer eligibility and threshold governance
3	`src/interfaces/IVapiStaking.sol`	133	Staking interface
4	`src/interfaces/IReviewersGate.sol`	100	Gate interface
5	`script/DeployVapiStaking.s.sol`	114	Validated deployment, staged record
6	`script/finalize-deployment-record.mjs`	-	Deployment record from receipts (staking path)

LoC counts physical lines; most of the growth since v1.1 is NatSpec. File hashes are listed in Appendix H.

In Scope

- `VapiStaking`: every function, including the checkpoint writes, the historical lookups and the new `cancelUnstake` and `custodyDeficit`.
- `ThresholdReviewersGate`: every function, including the threshold timelock with grace window and cancellation, the constructor probes and the two-step ownership.
- The interfaces `IVapiStaking` and `IReviewersGate`.
- The deployment script and the staking path of the receipt finalizer.
- The deployment runbook, reviewed for consistency with the code.
- The gaming resistance of the daily midnight snapshot, and the role of the gate in the Tasks reviewer selection.

Out of Scope

- The Tasks escrow stack (`EscrowV1`, `DisputeResolverV2`, `ReviewerRegistryV2`, `EscrowFactoryV1`, `FeeRouterV1`), covered by a separate report.
- `FeeRouter`, `BuybackBurner`, `FeeSplitter`, `FeeSplitterFactory`, and the retired V1 resolver and registry.

- The vAPI token, its Uniswap V4 hook and its router.
- The off-chain Compute control plane.
- Commit `3ed73a7d3` and any later commit (Appendix I).
- Frontend and backend application code.
- Third-party libraries, which are assumed correct and were checked for published advisories only.
- Economic and tokenomics modeling beyond the snapshot gaming analysis.

Methodology & Tools

Approach Overview

The audit combined manual review, static analysis, symbolic execution, stateful fuzzing and proof-of-concept tests. It followed the phases of NIST SP 800-115 and PTES, and was checked against the OWASP Smart Contract Top 10 (2025), the EEA EthTrust Security Levels, the SWC Registry and CWE.

For the remediation verification we:

1. Confirmed the package integrity against the audited commit.
2. Reviewed every changed line, then each contract function by function and as a whole.
3. Turned every proof of concept into a test of the safe behaviour.
4. Updated and re-ran the symbolic specifications and the fuzzing campaigns.
5. Re-ran Slither and Aderyn and triaged every result.
6. Executed the real-token fork tests.
7. Simulated the announced change of the initial threshold.

Techniques Applied

Technique	Applied
Manual code review	[x] Every changed line and every function
Static analysis	[x] Slither, Aderyn (Semgrep in v1.1)
Formal verification	[x] Partial: 18 bounded symbolic proofs
Symbolic execution	[x] Halmos (Mythril in v1.1)
Fuzzing	[x] Echidna, Medusa, Foundry invariants
Dependency analysis	[x] OpenZeppelin advisories, Solidity bug list
Threat modeling	[x] Custody, snapshot, governance, deployment
Gas review	[x] Appendix A

Tools Used

Tool	Purpose	Version	Re-run in v1.2
Slither	Static analysis	0.11.5	Yes
Aderyn	Static analysis	0.6.8	Yes
Semgrep	Pattern-based SAST	1.159.0	No
Mythril	Symbolic execution	0.24.8	No
Halmos	Symbolic proofs	0.3.3	Yes
Echidna	Fuzzing	2.3.3	Yes
Medusa	Fuzzing	1.5.1	Yes

Tool	Purpose	Version	Re-run in v1.2
Foundry	Tests and invariants	1.7.1	Yes

Review Process

1. **Kick-off and documentation review**
2. **Automated analysis** .
3. **Manual expert review**
4. **Exploitability validation**
5. **Client triage**
6. **Draft reporting**
7. **Remediation verification**
8. **Final delivery**

Severity & Status Definitions

House Standard Risk Model

House Severity	CVSS Base Range	Typical Business Impact
Critical	9.0 - 10.0	Immediate risk to funds, governance or integrity
High	7.0 - 8.9	Material risk, fix before release
Medium	4.0 - 6.9	Contained risk, fix on schedule
Low	0.1 - 3.9	Limited exploitability or impact
Informational	0.0	Hardening without direct security impact

CVSS v3.1 conventions:

- Attack Vector is Network.
- Privileges Required is Low for open roles and High for owner, deployer or issuer.
- Confidentiality is None.
- Integrity and Availability are High for loss of funds or of a core function, and Low for limited or recoverable effects.

Gas optimizations are not scored.

Status Lifecycle

Status	Definition
Open	Confirmed, not remediated
In Progress	Fix started, not merged
Resolved	Fix submitted by the client
Verified	Fix independently validated by SolidProof
Accepted Risk	Residual risk formally accepted by the client

Logical Understanding / Contract Architecture

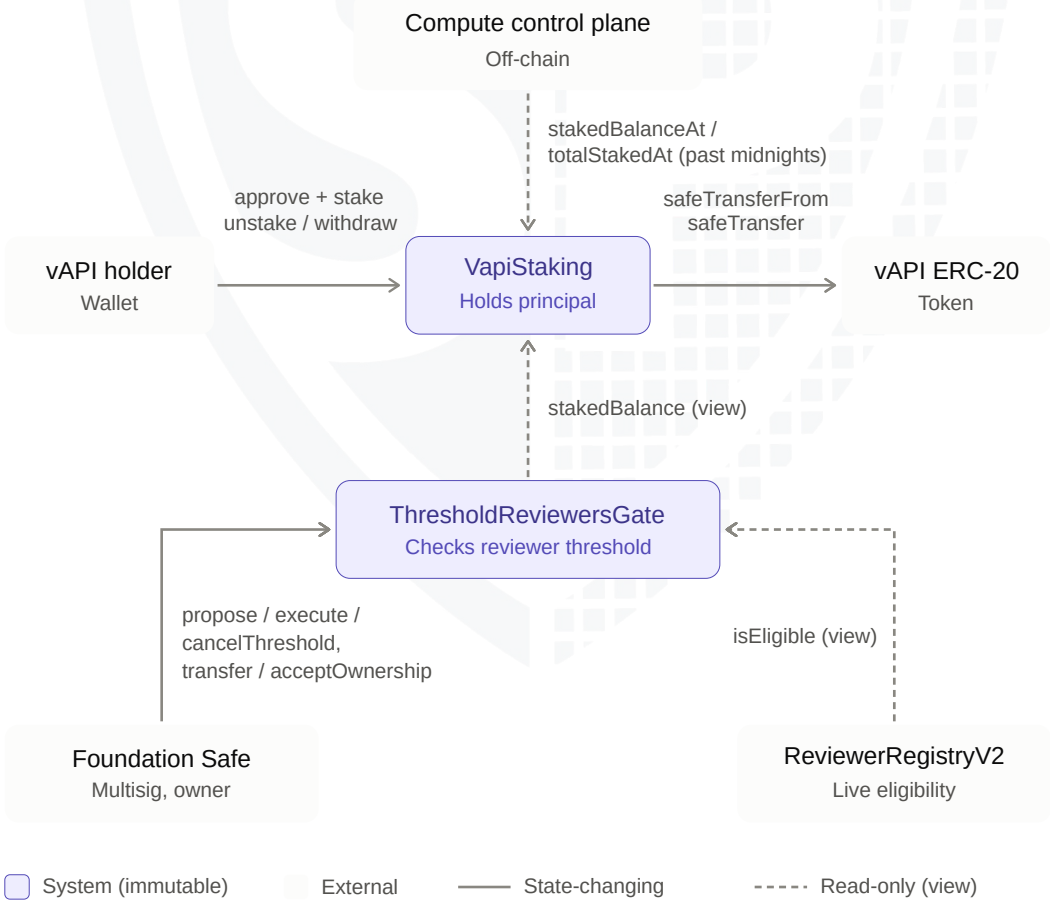
System Overview

The system consists of two immutable contracts:

- `VapiStaking` holds vAPI principal for stakers and records active balances in timestamp checkpoints.
- `ThresholdReviewersGate` reads the live active balance and answers whether a wallet meets the reviewer threshold.

Funds only flow between a staker's wallet and `VapiStaking`: in through `stake`, and out through `withdraw` to the wallet that requested the unstake. `cancelUnstake` moves no tokens. Data flows outward through read-only views, to the gate and the Tasks reviewer registry (live eligibility) and to the Compute control plane (balances at past midnights).

Contract Interactions & Data Flow



Contract	Inherits	Libraries	External calls
VapiStaking	IVapiStaking, ReentrancyGuardTransient	SafeERC20, Checkpoints, SafeCast	vAPI token only
ThresholdReviewersGate	IReviewersGate, Ownable2Step	SafeCast	stakedBalance (view); constructor probes

Entry points.

1. stake(amount) requires a resulting active balance of at least MIN_STAKE = 1e18. It updates the active balance, the total, the staker count and both checkpoints before the token is pulled.
2. requestUnstake(amount) moves an amount into a single pending record that unlocks after 604,800 seconds. The remaining active balance must be zero or at least MIN_STAKE.
3. cancelUnstake() moves the whole pending amount back to the active balance, before or after the cooldown, and writes checkpoints.
4. withdraw() pays the pending amount to msg.sender once block.timestamp >= availableAt.

Storage layout.

- VapiStaking uses slots 0 to 6, unchanged from v1.1, and keeps the reentrancy flag in transient storage.
- The gate uses slots 0 to 4: _owner, _pendingOwner, threshold, pendingThreshold and pendingThresholdValidAt.
- No proxy is involved, and both contracts are deployed fresh.

Snapshot integrity.

- A token that counted at any second can only leave through withdraw, which requires a pending request that did not count for the full seven days before.
- Cancelling deletes the request, and a new request starts a new cooldown.
- The same tokens therefore still cannot be counted for two wallets in one snapshot, and staking across midnight still locks the principal for at least seven days.

External Dependencies & Integrations

Dependency	Type	Risk	Notes
OpenZeppelin Contracts 5.6.1	Library	Low	Pinned; no advisory affects the used modules
vAPI token (`0x1eA8deB921AA6015f4938010C848140b98108Dd2`)	Staking token	Low	18 decimals; pinned by the script on Base mainnet
Foundation Safe	Gate owner	Low	Script requires a Safe threshold of at least two
`ReviewerRegistryV2`	Consumer of `isEligible`	Medium	Eligibility decides the Tasks dispute panels
Compute control plane	Consumer of historical views	Medium	Not reviewed; reads past seconds only
EIP-1153	EVM feature	Low	Transient storage, supported on Base

Code Overview

Metric	Value
Lines of code	769 physical lines in five Solidity files (v1.1: 338)
Contracts	2 contracts, 2 interfaces, 1 deployment script
External functions	16 in `VapiStaking`, 18 in the gate, all documented
OpenZeppelin imports	6 modules plus `IERC20Metadata` in the script

File (client suites, `--ir-minimum`)	Lines	Statements	Branches	Functions
`VapiStaking.sol`	100%	94.5%	66.7%	100%
`ThresholdReviewersGate.sol`	98.1%	98.5%	100%	100%
`DeployVapiStaking.s.sol`	90.3%	89.0%	80.0%	100%

Test suite assessment.

- **Our run:** 208 tests passed, 0 failed. This covers the client's 75 staking tests, the 7 fork tests, our 18 verification tests and the escrow and fee tests that don't import the staking contracts.
- **Client's reported count:** 233 passed. The delivered package lacked a few test-side adjustments needed to rebuild that exact count; they don't affect the staking contracts.
- **Fork tests:** with RPC URLs set, all seven executed and passed. Without an RPC they still return early instead of being reported as skipped.

Findings Overview

Summary Table

ID	Title	Severity	Status
STK-05	Deployment inputs are not validated	Low	Verified
STK-01	Threshold proposals never expire and cannot be withdrawn	Informational	Verified
STK-02	No minimum position size	Informational	Verified
STK-03	Historical lookups accept the current timestamp	Informational	Verified
STK-04	Single-step ownership with an available renounce path	Informational	Verified
STK-06	Interfaces carry no NatSpec	Informational	Verified
STK-07	A custody shortfall is reported as zero	Informational	Verified
STK-08	Unstake requests cannot be cancelled	Informational	Verified
STK-09	Compiler version with known issues	Informational	Verified

No new finding was raised in the remediation verification.

Detailed Findings

Descriptions and recommendations are kept as issued in v1.1, and line numbers refer to the audited commit. Each finding ends with the client's response and our verification of commit `716796dcf`.

STK-05: Deployment inputs are not validated

Field	Value
Severity	Low (raised from Informational in v1.1)
Status	Verified
Location	`script/DeployVapiStaking.s.sol:L9-L28`; `src/ThresholdReviewersGate.sol:L33-L38`
Category	Input Validation / Deployment
CVSS v3.1	3.3 (`AV:N/AC:H/PR:H/UI:N/S:U/C:N/I:L/A:L`)
CWE	CWE-20
Preconditions	Wrong `VAPI_TOKEN` or `FOUNDATION_MULTISIG` on Base mainnet
Impact	Immutable misconfiguration; a mintable token would hand out reviewer stake

Description

The script passed `VAPI_TOKEN` and `FOUNDATION_MULTISIG` from the environment straight into immutable constructors, and neither value had a default. The staking constructor only checked that the token address has code, and the gate constructor only checked that the staking address has code. On Base mainnet, a wrong token or a plain wallet as the gate owner would have been accepted without warning. The script also wrote the deployment record during its local execution, before the broadcast transactions were confirmed.

Impact

A wrong token, owner or gate invalidates the system and can only be repaired by redeploying. With the permissionlessly mintable test token used on Base Sepolia, anyone could mint stake and register as a Tasks reviewer. A record written before confirmation can list a deployment that failed or landed later.

Proof of Concept / Exploit Scenario

1. The operator runs the script on chain 8453 with `VAPI_TOKEN` still pointing at a test token, or with `FOUNDATION_MULTISIG` set to an EOA.
2. Both constructors accept the values, and the deployment record is written immediately.
3. With a mintable token, any address mints tokens, stakes the threshold and registers as an eligible Tasks reviewer.

Recommendation

Validate the inputs for mainnet in the script, write the record only from confirmed receipts, and verify the deployed bytecode against the audited commit. In the gate constructor, consider also checking that the staking contract answers `stakingToken()` and `cooldownSeconds()` as expected.

Client Response

```
Fixed in 716796dcf. The script validates inputs before
broadcast on every chain (token has code, 18 decimals, non-
zero owner). On 8453 it additionally requires the canonical
vAPI token 0x1eA8deB921AA6015f4938010C848140b98108Dd2, an
owner with code and getThreshold() >= 2. The record is no
longer written during simulation; finalize-deployment-
record.mjs writes it from mined receipts. The gate
constructor probes stakingToken() and cooldownSeconds().
```

Verification Method: We reviewed the script, the finalizer and the gate constructor.

- `validateInputs` runs before the broadcast. Both external calls are wrapped in `try`/`catch` with named errors.
- After the broadcast, the script asserts the wiring and writes only a pending record.
- The finalizer refuses a chain mismatch, any failed receipt and any address that doesn't match a successful creation receipt.
- The gate constructor rejects targets with failing calls, short or malformed return data, a token without code, or a cooldown other than seven days.

Tests: the client's 15 deployment tests pass, as do our gate-probe test and our finalizer checks (foreign address, failed receipt, wrong chain). The deployment fork test with the real token also passes on a Base mainnet fork.

Verification Result: Verified.

Residual Risk:

- The script checks that the owner is a Safe-like contract with a threshold of at least two, but not that it is the specific Foundation Safe. The runbook covers that check.
- The deployed bytecode match follows after the deployment.

STK-01: Threshold proposals never expire and cannot be withdrawn

Field	Value
Severity	Informational (reclassified from Low in v1.1)
Status	Verified
Location	`src/ThresholdReviewersGate.sol:L44-L66`
Category	Governance / Timelock
CVSS v3.1	0.0 (only the trusted owner can execute)
CWE	CWE-841
Preconditions	A pending proposal executed long after the announced date
Impact	Reviewers lose eligibility without fresh notice; no funds affected

Description

`proposeThreshold` stored a single `pendingThreshold` with `pendingThresholdValidAt = now + 7 days`, and `executeThreshold` applied it at any moment after that timestamp. Nothing ever invalidated a proposal: there was no expiry and no cancellation, and the only way to get rid of a pending value was to overwrite it.

Impact

A proposal that was announced, discussed and then left unexecuted remained executable forever, so it could take effect months later without a fresh notice period. Raising the threshold disqualifies reviewers from new panels in the same block.

Proof of Concept / Exploit Scenario

In v1.1, a proposal of 1,000,000 vAPI was executed 365 days after it was made, and a reviewer staked at 150,000 vAPI lost eligibility in that block.

Recommendation

Add an owner-only cancellation and a grace period after which a proposal expires and a fresh proposal with a new notice period is required.

Client Response

```
Fixed in 716796dcf. PROPOSAL_GRACE_PERIOD = 14 days;
executeThreshold() reverts ThresholdChangeExpired after
validAt + grace; cancelThreshold() (owner) clears a pending
or expired proposal and emits ThresholdChangeCancelled; new
view pendingThresholdExpiresAt().
```

Verification Method: We reviewed `executeThreshold`, `cancelThreshold` and `pendingThresholdExpiresAt`.

- Execution is only possible from `validAt` to `validAt + 14 days`, both ends inclusive.

- Cancellation is owner-only, works before and after expiry, clears both fields and leaves the active threshold unchanged.

The v1.1 proof of concept, inverted, now reverts with `ThresholdChangeExpired`, and the reviewer stays eligible. The boundary seconds are tested. The symbolic proofs cover every proposal value and every elapsed time up to ten years.

Verification Result: Verified.

Residual Risk: None. An expired proposal stays readable until it is cancelled or replaced, as documented.

STK-02: No minimum position size

Field	Value
Severity	Informational (reclassified from Low in v1.1)
Status	Verified
Location	`src/VapiStaking.sol:L72-L91` (and `L94-L124`)
Category	Resource Consumption
CVSS v3.1	0.0
CWE	CWE-770
Preconditions	Many fresh addresses stake one wei each
Impact	Misleading staker count, more wallets for the daily snapshot

Description

`stake` accepted any non-zero amount, and a wallet moving from zero to a positive balance incremented `stakerCount`. `requestUnstake` likewise allowed leaving a one-wei remainder.

Impact

On chain, dust positions only inflated the staker count. The documented daily job reads every known staker, so many dust positions multiply its reads.

Proof of Concept / Exploit Scenario

In v1.1, 250 one-wei stakes yielded `stakerCount == 250` with a total stake of 250 wei.

Recommendation

Introduce a minimum active position, for example `MIN_STAKE = 1e18`, in `stake` and in `requestUnstake`, so that a remaining active balance is either zero or at least the minimum. Off chain, skip wallets that cannot earn a limit and read balances in batches.

Client Response

```
Fixed in 716796dcf. MIN_STAKE = 1e18; stake reverts
BelowMinimumStake when the resulting balance is below it;
requestUnstake reverts when a non-zero remainder would be
below it (full exit allowed).
```

Verification Method: We reviewed `stake`, `requestUnstake` and `cancelUnstake`. A cancellation cannot create a position below the minimum, because a pending amount with a zero active balance always comes from a full exit of a valid balance.

The v1.1 proof of concept, inverted, now makes all 250 one-wei stakes revert. The minimum rule is proven symbolically, and the property "every active balance is zero or at least one vAPI" held under both fuzzers and the Foundry invariants.

Verification Result: Verified.

Residual Risk: Each counted wallet now has to lock at least one vAPI. Batched reads in the off-chain Compute job are outside the reviewed code.

STK-03: Historical lookups accept the current timestamp

Field	Value
Severity	Informational
Status	Verified
Location	`src/VapiStaking.sol:L160-L169, L179-L182`
Category	Time Handling / Data Finality
CVSS v3.1	0.0
CWE	CWE-367
Preconditions	An integrator reads the current block timestamp
Impact	A balance that can still change within the same block

Description

`_validateLookupTime` reverted only for `timestamp > block.timestamp`. All writes inside one second share one checkpoint key, so the value at the current timestamp is not final until the block is complete.

Impact

An integrator reading the current second could be shown a stake that is requested back within the same block. The documented Compute consumer reads past midnights and was not affected.

Proof of Concept / Exploit Scenario

In v1.1, a lookup returned 100,000 vAPI and then 0 for the same timestamp within one block.

Recommendation

Reject lookups at the current second, or document that only past timestamps are final.

Client Response

```
Fixed in 716796dcf. _validateLookupTime rejects timestamp
>= block.timestamp (FutureLookup). The Compute snapshot job
reads only safe blocks strictly after the snapshot second.
```

Verification Method: We reviewed both historical views, which now reject the current second, and the NatSpec states the rule. The inverted proof of concept shows that the current second reverts and that the same second returns its final value one second later. This is covered by the symbolic proofs and by a negative fuzzing action.

Verification Result: Verified.

Residual Risk: None. No on-chain consumer uses the historical views; the gate uses the live balance.

STK-04: Single-step ownership with an available renounce path

Field	Value
Severity	Informational
Status	Verified
Location	`src/ThresholdReviewersGate.sol:L11, L33`
Category	Access Control
CVSS v3.1	0.0
CWE	CWE-269
Preconditions	Mistyped transfer or accidental renounce by the owner
Impact	Threshold frozen permanently

Description

The gate inherited `Ownable`, so `transferOwnership` took effect immediately and `renounceOwnership` remained callable. The Tasks deployment relies on the gate owner being the escrow council, which was only checked once at deployment.

Impact

A mistyped transfer or an accidental renounce would have frozen the threshold permanently.

Recommendation

Use a two-step transfer and disable renouncing. Re-check the gate owner against the council whenever either role changes.

Client Response

```
Fixed in 716796dcf. Gate is Ownable2Step;
renounceOwnership() reverts RenounceDisabled; interface
exposes pendingOwner, transferOwnership and
acceptOwnership.
```

Verification Method: We reviewed the inheritance and the overrides. Ownership changes only when the pending owner accepts, the current owner can replace or clear a pending transfer, and `renounceOwnership` always reverts. This is unit-tested and proven symbolically for every pair of addresses.

Verification Result: Verified.

Residual Risk: The gate owner and the escrow council must be kept equal whenever either role changes.

STK-06: Interfaces carry no NatSpec

Field	Value
Severity	Informational
Status	Verified
Location	`src/interfaces/IVapiStaking.sol:L4-L28`; `src/interfaces/IReviewersGate.sol`
Category	Documentation
CVSS v3.1	0.0
CWE	CWE-1059
Preconditions	Not applicable
Impact	Integrators could misread semantics

Description

Every function of `VapiStaking` pointed to the interface documentation through `@inheritdoc`, but the interfaces had none, so the generated documentation was empty. Integrator-relevant rules lived only in external documents.

Recommendation

Document each interface function with `@notice`, `@param` and `@return`, including units, revert conditions and edge cases.

Client Response

> `Fixed in 716796dcf. Full NatSpec on IVapiStaking and IReviewersGate; contracts use @inheritdoc.`

Verification Method: We checked the compiler's documentation output. All 34 functions, all custom errors and all events are documented. Units, bounds, revert conditions, the lookup rule and the grace window match the code.

Verification Result: Verified.

Residual Risk: The documented initial threshold of 100,000 vAPI must be updated if the starting value changes.

STK-07: A custody shortfall is reported as zero

Field	Value
Severity	Informational
Status	Verified
Location	`src/VapiStaking.sol:L143-L147`
Category	Monitoring
CVSS v3.1	0.0
CWE	CWE-754
Preconditions	Token balance below recorded principal (impossible with vAPI)
Impact	Monitoring would miss an insolvency

Description

`unaccountedStakingToken` returned zero whenever the balance was below `totalStaked + totalPendingUnstake`, so a shortfall was indistinguishable from a balanced state.

Recommendation

Expose the shortfall explicitly. Optionally, compare the received amount with `amount` in `stake`.

Client Response

> `Fixed in 716796dcf. custodyDeficit() view next to unaccountedStakingToken().`

Verification Method: We reviewed the new view, which mirrors the surplus view. A test with a token that allows balances to be clawed back reports the shortfall correctly. The property "no deficit" held under both fuzzers, and the real-token fork test passes.

Verification Result: Verified.

Residual Risk: None. The optional received-amount check was not adopted, which is acceptable because the token is pinned on mainnet.

STK-08: Unstake requests cannot be cancelled

Field	Value
Severity	Informational
Status	Verified
Location	`src/VapiStaking.sol:L94-L124`
Category	Usability
CVSS v3.1	0.0
CWE	N/A
Preconditions	A staker requests an unstake by mistake
Impact	Seven days without reviewer eligibility

Description

A request took effect immediately and could only be undone by withdrawing after seven days and staking again.

Recommendation

Add a cancellation that moves the pending amount back into the active balance and writes a new checkpoint.

Client Response

> `Fixed in 716796dcf. cancelUnstake() restores the pending principal to active stake, writes checkpoints, emits UnstakeCancelled; no token movement.`

Verification Method: We reviewed `cancelUnstake`. It is `nonReentrant`, makes no external call, touches only the caller's own record, restores the staker count and writes both checkpoints.

Tests show that eligibility returns, no token moves, earlier history is not rewritten, and a new request restarts the full cooldown. The function is proven symbolically, fuzzed against an exact expected state, and was run on a Base Sepolia fork.

Verification Result: Verified.

Residual Risk: None in staking. One consequence for the escrow side is described in Appendix I.

STK-09: Compiler version with known issues

Field	Value
Severity	Informational
Status	Verified
Location	`src/VapiStaking.sol:L2`, `src/ThresholdReviewersGate.sol:L2`, interfaces
Category	Compiler
CVSS v3.1	0.0
CWE / SWC	CWE-1104 / SWC-102
Preconditions	A trigger condition of a known 0.8.35 bug (none present)
Impact	Theoretical

Description

Solidity 0.8.35 was affected by several known compiler bugs, two of them of medium severity. None of their trigger conditions occurred in this code.

Recommendation

Upgrade to 0.8.37 or newer, keep the version pinned and re-run the full suite.

Client Response

> `Fixed in 716796dcf. VapiStaking, ThresholdReviewersGate and IVapiStaking pinned to 0.8.37; IReviewersGate at >=0.8.35 <0.9.0 because the 0.8.35 registries import it. The Tasks contracts keep 0.8.35 with identical runtime bytecode.`

Verification Method: We reviewed the pragmas. The version range on `IReviewersGate` is justified for an interface without bytecode. We compiled with the official 0.8.37 binary and re-ran the tests, proofs and fuzzing on the via-IR build. The runtime bytecode of all eight escrow and registry contracts is identical to the audited build.

Verification Result: Verified. 0.8.37 is the newest released compiler.

Residual Risk: A newly listed low severity optimizer bug (SOL-2026-7) affects every release from 0.8.19 to 0.8.37, and its fix in 0.8.38 is not released yet. Testing on the compiled code showed no deviation, so no action is needed.

Centralisation, Access Control & Privilege Analysis

Overview

`VapiStaking` has no privileged role, and no key can move, freeze or seize staked vAPI. The only privileged actor in scope is the gate owner, the Foundation Safe, and its power is limited to the reviewer threshold:

- The threshold is bounded between 10,000 and 1,000,000 vAPI.
- A change takes effect only after seven days of notice and within a 14-day window.
- Ownership moves only when the new owner accepts, and it cannot be renounced.

Because the threshold decides who may sit on Tasks dispute panels, it remains a security parameter of the dispute process.

On-Chain Roles & Permissions

Role	Permissions	Timelock	Multisig	Risk
Gate owner	Propose, execute, cancel threshold; start ownership transfer	7-day notice, 14-day window; transfer needs acceptance	Enforced by script on 8453	Low
Pending owner	`acceptOwnership`	-	-	Low
Staking admin	None	-	-	None
Any wallet	Stake, unstake, cancel, withdraw own position	-	-	Low

Upgradeability Analysis

Contract	Upgrade Pattern	Admin	Risk
`VapiStaking`	None (immutable)	None	None
`ThresholdReviewersGate`	None (immutable)	Gate owner, threshold only	Low

Centralisation Risk Summary

Risk	Status	Recommendation
Single-step ownership and renounce	Resolved (STK-04)	-
Stale threshold proposals	Resolved (STK-01)	-
Owner not enforced to be a Safe	Resolved (STK-05)	Confirm the Safe's signers at deployment
Threshold controls the reviewer pool	Open, escrow side	Size it together with the escrow fixes

Conclusion

Summary

The staking contracts were small, well structured and conservative before the remediation, and they remain so after it. All nine findings were fixed as recommended or in an equivalent way, and every fix was verified by code review, inverted proofs of concept, symbolic proofs and fuzzing. The new functions introduce no new vulnerability. Custody remains out of reach of any privileged party, every write is reentrancy-guarded, and the seven-day lock still protects the midnight snapshot.

Positive Observations

- `VapiStaking` has no owner, pause, upgrade, sweep or rescue path.
- Every write is reentrancy-guarded and follows checks-effects-interactions; `cancelUnstake` makes no external call at all.
- The remediation follows the recommendations closely, uses OpenZeppelin `Ownable2Step` and comes with targeted tests for every fix.
- The deployment path fails closed: wrong inputs abort before broadcast, and the record only comes from matching receipts.
- 18 of 18 symbolic specifications passed, and 12 stateful properties held across about 9.3 million fuzzed calls.

Decision Rationale: The rating is Secure because no critical, high or medium severity finding exists, all findings are verified as fixed, and the custody guarantees were proven again on the remediated code. Release readiness still depends on two things: the deployment commit must match the verified commit or be reviewed, and the deployed bytecode must be shown to match it.

Appendix A - Gas Optimizations and Code Quality Notes

Gas Optimizations

ID	Location (v1.2)	Suggestion	Status
STK-G1	`VapiStaking.sol` L120-L121, L147-L159, L179-L181	Keep the new `totalStaked` in a local variable	Not addressed
STK-G2	`ThresholdReviewersGate.sol` L30-L32	Pack pending threshold and timestamp into one slot	Not addressed
STK-G3	`VapiStaking.sol` L229, L235	Use `upperLookupRecent`	Not addressed
STK-G4	`VapiStaking.sol` L41-L44	Pack `totalStaked` and `totalPendingUnstake`	Not addressed

Code Quality Notes

- **NatSpec:** complete.
- **Naming:** consistent; constant getters use upper case.
- **Events:** every state change emits, except the initial threshold set in the constructor.
- **unchecked blocks:** three, all provably safe.
- **Pragma:** exact `0.8.37`, except the justified range on `IReviewersGate`.
- **Dead code, unused storage, magic numbers:** none.

Appendix B - Claims Check

Claim	Verdict	Explanation
Can Burn	**No**	-
Can Lock	**Yes (user-initiated only)**	7-day cooldown after a request; cancellable; no privileged lock
Can Mint	**No**	-
Can Set Fees (higher than 25%)	**No**	No fees
Can Blacklist	**No**	-
Is Upgradeable	**No**	No proxy or delegatecall
Can Pause / Has Emergency Stop	No	-
Has Timelock on Admin Functions	Yes	Threshold: 7-day notice, 14-day window
Has Two-Step Ownership Transfer	Yes	`Ownable2Step`
Owner is Multisig	Enforced by script	Confirm on chain after deployment
Has Renounceable Admin Roles	No	Renounce always reverts
Has Fee-on-Transfer	No	Plain-token assumption documented
Has Anti-Whale / Max-TX / Max-Wallet Limits	No	Minimum position of one vAPI only
Has Trading-Enable Flag	No	-
Has Hidden Mint	No	Credits follow transfers or restore own pending principal
Has Reflection / Auto-Liquidity	No	-
Uses External Oracle	No	-
Has Snapshot / Voting Power	Yes (historical balances)	Past seconds only; no delegation
Implements Permit / EIP-712	No	-
Implements ERC-1363 / ERC-777 callbacks	No	Reentrancy-guarded
Has Anti-Bot / Anti-Sniper Logic	No	-
Has Reward / Staking Hooks	Conditional	No on-chain rewards; read by gate and Compute
Has Cross-Chain Bridge	No	-

Appendix C - Analysis Coverage by Category

Category	Result
Centralization	Staking has none; gate owner bounded, timelocked, two-step
Initialization	No proxy; the gate constructor validates its target
Events	Complete except the initial threshold
MEV / front-running	Not applicable; snapshot protected by the 7-day lock
Signatures / replay	Not applicable
Denial of service	No user loops; logarithmic lookups; dust prevented
Reentrancy	Guarded writes, checks-effects-interactions
Arithmetic	Checked; three safe `unchecked` blocks; no division
Oracles	Not applicable



Solidproof.io - Trust Made in Germany

Blockchain Security | Smart Contract Audits | KYC | Marketing

FutureVisions Deutschland UG (haftungsbeschränkt)

Heideland-Ost 34
24976 Handewitt
Germany, Deutschland

Sitz der Gesellschaft / Amtsgericht: Flensburg
HRB 16612 FL
USt-IdNr. DE364783510
Geschäftsführer: Mails Nielsen, Kevin Arens

Solidproof.io is a product of
FutureVisions Deutschland

Please note: You automatically accept our Terms and Conditions and Privacy Policy when using our services.

You can find the current version on our website: https://solidproof.io.

© 2026 SolidProof.io. All rights reserved. This document may not be reproduced or distributed without written consent.


Solid Proofed
Blockchain Security | Smart Contract Audits | KYC
Development | Marketing
MADE IN GERMANY